

Spring Stereotype Annotations

1 What are stereotype annotations?

Stereotype annotations tell Spring:

"This class is a component that should be managed by the Spring IoC container."

Common stereotypes:

`@Component` `@Service` `@Repository` `@Controller` `@RestController`

Spring detects these during **component scanning** and creates Spring Beans from them.

2 @Component

Generic stereotype.

```
@Component
public class EmailUtil {

    public void sendEmail() {
    }
}
```

Use `@Component` when the class doesn't specifically belong to the Controller, Service, or Repository layer.

INTERVIEW ANSWER

"@Component is a generic stereotype annotation used to register a class as a Spring Bean."

3 @Service

Used for the business/service layer.

```
@Service
public class UserService {

    public User getUser(Long id) {
        // business logic
    }
}
```

`@Service` is technically a specialization of `@Component`.

`@Component` ← `@Service`

The main benefit is **semantic clarity**—it tells developers that this class contains business logic.

INTERVIEW ANSWER

"@Service is a specialization of @Component used to indicate that a class contains business or service-layer logic."

4 @Repository

Used for the data-access layer.

```
@Repository
public class UserRepository {

    public User findUser(Long id) {
        // database operation
    }
}
```

Important additional feature:

Exception Translation: `@Repository` participates in Spring's persistence exception translation. For example, a database-specific exception can be translated into Spring's `DataAccessException` hierarchy.

INTERVIEW ANSWER

"@Repository marks a class as a persistence/data-access component and enables Spring's persistence exception translation."

5 @RestController

Used to create REST API controllers.

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getUser(id);
    }
}
```

@RestController = @Controller + @ResponseBody

Therefore, the return value is written directly to the HTTP response body. For example, returning a `user` object becomes JSON through Spring's HTTP message conversion (typically Jackson):

```
{
  "id": 10,
  "name": "Avnish"
}
```

6 Typical Spring Boot Architecture

You should be able to explain this architecture in an interview (Separation of Concerns):

HTTP Request → @RestController → @Service → @Repository → Database

```
@RestController
class UserController {
    private final UserService userService;

    @GetMapping("/users/{id}")
    User getUser(@PathVariable Long id) {
        return userService.getUser(id);
    }
}

@Service
class UserService {
    private final UserRepository userRepository;

    User getUser(Long id) {
        // Business logic
        return userRepository.findById(id);
    }
}

@Repository
class UserRepository {
    // Data access
}
```

7 @Controller vs @RestController

@Controller: Usually used for MVC applications where the controller returns a view.

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home() {
        return "home"; // View name
    }
}
```

@RestController: Used primarily for REST APIs. Returns the object directly as the response body.

```
@RestController
public class UserController {

    @GetMapping("/users")
    public User getUser() {
        return user; // Serialized Data
    }
}
```

```
}  
}
```

INTERVIEW ANSWER

"@RestController is effectively @Controller plus @ResponseBody, so its methods return data directly in the HTTP response body rather than resolving a view."

8 Are @Service and @Repository really necessary?

Technically, you could use `@Component` for many classes. But:

- `@Service` communicates: "This is business logic."
- `@Repository` communicates: "This is data-access logic."

They improve architecture, readability, maintainability, and `@Repository` adds persistence exception-translation behavior.

9-13 Interview Cross-Questions

Q9. How does Spring find these classes?

Through **component scanning**. Spring Boot's `@SpringBootApplication` includes `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. It searches packages for stereotype classes and registers them as Spring Beans.

Q10. What happens after Spring finds them?

Spring's IoC container / `ApplicationContext` creates and manages their bean instances. Another bean can then inject it (preferably via **constructor injection**):

```
private final UserService userService;  
  
public UserController(UserService userService) {  
    this.userService = userService;  
}
```

Q11. What is the difference between @Component and @Service?

Technically, both register a Spring Bean. Conceptually, `@Component` is a generic component, while `@Service` indicates a business/service component with clearer architectural intent.

Q12. Why use @Repository instead of @Component?

1. **Semantic clarity:** Identifies the persistence layer.
2. **Exception translation:** Translates database-specific exceptions into Spring's `DataAccessException` hierarchy.

Q13. Are these annotations creating objects?

Not directly in the standard Java sense. Spring's IoC container creates and manages the bean instances after discovering these component classes. This connects to: *IoC* → *Dependency Injection* → *Spring Beans*.

14 Master Interview-Ready Answer

If the interviewer asks: **"Explain Spring stereotype annotations."** Give this concise response:

"Spring stereotype annotations identify classes that should be managed as Spring Beans. **@Component** is the generic stereotype, **@Service** is used for business logic, **@Repository** is used for data-access logic and also provides persistence exception translation, and **@Controller** / **@RestController** is used for the presentation/API layer. Spring discovers these classes through component scanning and registers them in the ApplicationContext."

Key Follow-Up Checklist to Remember:

- @Controller vs @RestController
- @Component vs @Service
- Why use @Repository? (Exception Translation)
- How component scanning works (@ComponentScan)
- IoC vs Dependency Injection & Constructor Injection preference